

Challenge National Datavis BUT SD

Stratégie "Zéro Bug" : Architecture Python vers HTML Statique

Le Nouveau Paradigme : Il n'y a plus de serveur. Python sert uniquement d'usine. Vous exécutez un script `build.py` qui compile le travail de tout le monde et recrache un fichier `rendu_final.html` indestructible.

1. La Règle d'Or (Inchangée) : Le Contrat de Données

La règle des 15 premières minutes tient toujours. Vous devez définir les colonnes du CSV final avant de coder. Sans ça, impossible d'avancer en parallèle.

2. L'Architecture "Constructeur HTML"

Pour travailler à 5, on garde l'idée d'un fichier par personne, mais au lieu d'afficher le graphique, votre fichier va le **convertir en texte HTML**.

📁 Structure du projet Git

- `template.html` : Le squelette visuel vide (votre design, vos onglets HTML/CSS natifs). Fixe
- `build.py` : Le script chef d'orchestre. Il lit le template, appelle vos fichiers, injecte vos graphiques, et sauvegarde le tout. Exécuté à la fin
- `partie_milan.py` : Ton espace. Tu crées tes figures Plotly et tu les retournes sous forme de blocs HTML. Modifiable
- `style.css` : Pour le design de la page.

3. Le Code : Comment ça marche ?

A. Ton fichier (`partie_milan.py`)

Ton objectif n'est plus de faire `st.plotly_chart()`, mais d'utiliser `fig.to_html()`.

```

import pandas as pd
import plotly.express as px

def generer_html(df):
    # 1. Tu crées ton graphique Plotly normalement
    fig = px.bar(df, x='LAT', y='RRAB', title="Précipitations")

    # 2. MAGIE : Tu le transformes en bloc HTML brut
    # full_html=False -> ne génère que la div du graphique, pas la page entière
    # include_plotlyjs=False -> on le mettra une seule fois dans le template principal pour alléger
    div_html = fig.to_html(full_html=False, include_plotlyjs=False)

    # Tu peux ajouter tes métriques encapsulées dans du HTML basique
    kpi_html = f"
Moyenne : {df['RRAB'].mean():.1f}
"

    # 3. Tu retournes le tout
    return kpi_html + div_html

```

B. Le Chef d'Orchestre (build.py)

Ce script assemble les pièces du puzzle comme des Lego.

```

import partie_milan
import pandas as pd

# 1. On charge la donnée propre
df = pd.read_csv("data_propre.csv")

# 2. On récupère le code HTML généré par Milan
html_milan = partie_milan.generer_html(df)

# 3. On lit le template vide
with open("template.html", "r", encoding="utf-8") as f:
    page_entiere = f.read()

# 4. On remplace une balise invisible par ton travail
page_entiere = page_entiere.replace("", html_milan)

# 5. On sauvegarde le rendu final
with open("rendu_final.html", "w", encoding="utf-8") as f:
    f.write(page_entiere)

print("✅ Fichier rendu_final.html généré avec succès !")

```

4. Le Squelette (template.html) à coder d'avance

Dès ce soir, vous pouvez préparer un beau fichier HTML qui intègre Bootstrap (pour faire des colonnes et des onglets facilement sans coder 500 lignes de CSS) et la librairie Plotly.js.

```

<!DOCTYPE html>
<html>
<head>
  <title>Dashboard BUT SD</title>
  <!-- Bootstrap pour le design -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
  <!-- Plotly JS obligatoire pour afficher les graphiques -->
  <script src="https://cdn.plot.ly/plotly-latest.min.js"></script>
</head>
<body>
  <div class="container mt-5">
    <h1>Analyse des données</h1>

    <div class="section-milan">
      <h2>Partie 1 : Milan</h2>
      <!-- INJECTER_MILAN -->
    </div>

  </div>
</body>
</html>

```

5. Le Flux de travail Git (Identique)

1. Chacun code dans sa branche (branche_milan) et utilise son Mock Data.
2. Pour tester, tu lances `python build.py` et tu ouvres `rendu_final.html` dans Chrome.
3. Quand tout est bon, vous fusionnez sur main. Le dernier `python build.py` créera le livrable parfait à envoyer au jury.